

Aula 01

DataPrev (Perfil 3: Desenvolvimento de Software) Desenvolvimento de Software - 2026 (Pós-Edital)

Autor:
Vinicius Borges

09 de Julho de 2026

Índice

1) Paradigma Orientado a Objetos - Teoria	3
2) Paradigma Orientado a Objetos - Questões Comentadas	16
3) Paradigma Orientado a Objetos - Lista de Questões	25

ORIENTAÇÃO A OBJETOS

Conceitos Gerais

ATENÇÃO! Essa aula explora um **paradigma de programação**. Paradigmas de programação são **modelos ou estilos distintos de desenvolvimento de software** que estabelecem **princípios e técnicas para a organização e estruturação do código**. Eles definem abordagens específicas para a resolução de problemas computacionais, influenciando a forma como os algoritmos são construídos e como os dados são manipulados.

O Paradigma Orientado a Objetos (POO), ou Programação Orientada a Objetos, é um **modelo de programação** que se concentra na **criação de objetos que possuem características e comportamentos específicos**. É um dos paradigmas mais amplamente utilizados na programação de computadores atualmente, e é a base para muitas linguagens de programação populares, incluindo Java, Python, C++ e Ruby.

Em termos simples, a POO se concentra em **dividir um programa em objetos individuais, que podem ser usados e reutilizados em diferentes partes do programa**. Cada objeto tem sua própria identidade única e é capaz de executar ações específicas.

Enquanto na programação tradicional usamos dados e funções que os manipulam, na programação orientada a objetos tratamos o sistema em **objetos** (que são compostos por *atributos*), **métodos** e **classes**.



Bom, você deve estar se perguntando “mas que diabos é objetos, classe métodos?” haha então vamos tentar entender melhor o que é cada um desses.

Só um adendo, para facilitar o andamento da aula e o seu entendimento sobre o assunto, vamos criar um objeto na linguagem **Java** – já que a maioria das bancas, quando cobram POO associado a alguma linguagem, costumam cobrar associada a ela. Não se preocupe caso você não conheça a linguagem, vou explicar o que for necessário para você entender o contexto.

Classes

Uma classe é um tipo de estrutura fundamental no paradigma orientado a objetos. Ela serve como um modelo ou uma descrição para criar objetos. Em termos mais simples, uma **classe é como uma representação abstrata de uma instância da vida real, que define os atributos (variáveis) e os métodos (funções) que um objeto desse tipo terá**. Pense em uma classe como uma representação genérica de um objeto, que define quais devem ser seus atributos e métodos, mas sem atribuir valor a eles.

Podemos trabalhar com alguns tipos de classes principais:

Classes Abstratas

Uma **classe abstrata** é uma classe que **não pode ser instanciada diretamente**, ou seja, **você não pode criar objetos diretamente a partir de uma classe abstrata**. Ela serve como uma "base" para outras classes que herdam dela, fornecendo uma estrutura comum e definindo métodos e atributos que as subclasses devem implementar. Uma classe abstrata pode conter métodos abstratos (métodos que não têm implementação na classe base) que as subclasses concretas devem obrigatoriamente implementar. Classes abstratas são usadas para criar hierarquias de classes e promover a reutilização de código.

Classes Concretas

Uma **classe concreta** é uma classe que **pode ser instanciada diretamente**, ou seja, você pode criar objetos diretamente a partir dela. Ela pode herdar de classes abstratas ou outras classes concretas, implementando assim os métodos definidos nas classes pai (através de herança) ou possuindo seus próprios métodos. Classes concretas são utilizadas para criar objetos e encapsular comportamentos e atributos específicos.

Classes Estáticas

Uma **classe estática não pode ser instanciada**. Ela é uma classe que contém métodos e atributos estáticos, ou seja, aqueles que pertencem **apenas** à própria classe, não a instâncias individuais da classe. Métodos estáticos podem ser chamados diretamente na classe, sem a necessidade de criar um objeto. Eles são frequentemente usados para funcionalidades que não dependem do estado individual de um objeto.

Visibilidade

Além dos tipos de classes, podemos ter diferentes visibilidades para uma classe - isso é, diferentes permissões para quem, ou o que, pode acessar ela, no contexto do sistema. As visibilidades são: **privada** (private), **pública** (public) ou **protegida** (protected).

- **Pública** - uma classe declarada como **pública é acessível de qualquer lugar**, em qualquer pacote (package) e por qualquer classe. Isso significa que a classe pode ser usada em qualquer parte do programa, independentemente do local em que está sendo acessada.

- **Protegida** - classes protegidas podem ser **acessadas apenas no mesmo pacote e por subclasses** (classes que herdam da classe protegida). Isso é útil quando você deseja que uma classe seja estendida, mas não acessada diretamente fora do pacote.
- **Privada** - as classes privadas são **acessíveis apenas na própria classe** em que foram definidas. Elas não podem ser acessadas por outras classes, mesmo que estejam no mesmo pacote. Geralmente, classes privadas são usadas para encapsular detalhes de implementação e não são visíveis fora da classe em que são declaradas.

Bom, vamos agora ao nosso exemplo. Primeiramente, vamos criar uma **classe abstrata VEÍCULO**, em Java:

Java

```
abstract class Veiculo {  
    protected String marca;  
    protected String modelo;  
    protected int ano;  
    protected String cor;  
  
    public Veiculo(String marca, String modelo, int ano, String cor) {  
        this.marca = marca;  
        this.modelo = modelo;  
        this.ano = ano;  
        this.cor = cor;  
    }  
    public abstract void ligar();  
    public abstract void desligar();  
}
```

Alguns detalhes do código para você não ficar perdido:

- Estamos criando a classe abstrata a partir do comando **abstract class**.
- Definimos o tipo de dado que vai em cada atributo - por exemplo, **marca** é do tipo **string** (texto). O **protected** é responsável por lidar com a visibilidade da classe.
- Em **public Veiculo** estamos definindo um construtor que irá, de fato, criar a nossa classe pública. Damos quatro parâmetros para esse construtor - que são nossos quatro atributos.
- Também estamos criando dois **métodos** (não se preocupe, que em alguns momentos você saberá o que são), *ligar* e *desligar*.

Bom, temos nossa classe abstrata. Agora, vamos criar uma classe concreta **Carro**, que irá herdar os atributos de sua classe "pai", Veículo.

Java

```
class Carro extends Veiculo {  
    public Carro(String marca, String modelo, int ano, String cor) {  
        super(marca, modelo, ano, cor);  
    }  
    public void ligar() {  
        System.out.println("Carro ligado.");  
    }  
    public void desligar() {  
        System.out.println("Carro desligado.");  
    }  
}
```

Através do comando **extends**, estamos criar uma classe que herda os atributos e métodos da classe **Veiculo**. No lugar dos métodos, estamos criando um *print*, que é uma mensagenzinha que irá retornar toda vez que executarmos o comando - se executarmos o **ligar**, teremos "Carro ligado", e se executarmos **desligar**, "Carro desligado".

Pronto, temos nossas classes, podemos seguir aos **objetos**.

Objetos

Um **objeto é uma instância específica de uma classe**. Ele representa um item do mundo real que pode ser manipulado dentro do programa. Quando você cria um objeto, está utilizando o modelo (classe) para criar uma entidade com valores específicos para seus atributos. Cada objeto possui seu próprio conjunto de dados e pode executar os métodos definidos em sua classe.

Vamos criar dois objetos:

- OBJETO 1:
 - Marca = Toyota
 - Modelo = Corolla
 - Ano = 2022
 - Cor = Prata
- OBJETO 2:
 - Marca = Ford
 - Modelo = F150
 - Ano = 2023
 - Cor = Cinza

Java

```
class Carro extends Veiculo {  
    public Carro(String marca, String modelo, int ano, String cor) {  
        super(marca, modelo, ano, cor);  
    }  
    public void ligar() {  
        System.out.println("Carro ligado.");  
    }  
    public void desligar() {  
        System.out.println("Carro desligado.");  
    }  
    //Construtor para criarmos o objeto  
    public static void main(String[] args) {  
        // Criando o primeiro objeto Carro  
        Carro carro1 = new Carro("Toyota", "Corolla", 2022, "Prata");  
        // Criando o segundo objeto Carro  
        Carro carro2 = new Carro("Ford", "F150", 2023, "Cinza");  
    }  
}
```



Aqui, eu trouxe novamente o código pois estamos criando o objeto **dentro** da classe Carro. Para isso, usamos o construtor **main**, que faz uma "extensão" da classe para suportar os novos objetos e cria-los.

Também poderíamos criar uma nova classe e criar os objetos dentro dela, sem necessidade do *main*.

Atributos

Atributos, também conhecidos como propriedades ou variáveis de instância, **são características ou dados que definem as propriedades de um objeto**, sendo definidos na sua classe.

Eles são as informações que uma instância específica de uma classe armazena, como nome, idade, endereço, altura, peso etc. Cada objeto criado a partir de uma classe possui seus próprios valores para seus atributos. No nosso exemplo, os atributos de carro são "marca", "modelo", "ano" e cor.

Métodos

Métodos são funções associadas a uma classe ou a um objeto específico da classe. Eles definem as ações que os objetos da classe podem executar. Os **métodos permitem que os objetos realizem tarefas**, manipulem seus atributos e interajam com outros objetos.

Vou trazer nosso último código, omitindo os objetos, para explorarmos os métodos.

Java

```
class Carro extends Veiculo {  
    public Carro(String marca, String modelo, int ano, String cor) {  
        super(marca, modelo, ano, cor);  
    }  
    public void ligar() {  
        System.out.println("Carro ligado.");  
    }  
    public void desligar() {  
        System.out.println("Carro desligado.");  
    }  
}
```

Temos dois métodos, que nada mais são que funções - ações que executam alguma coisa. No nosso caso, ligar e desligar apenas retornarão mensagens, como "Carro Ligado" e "Carro desligado". O comando **void** é usado para indicar que o método não retorna um valor específico - não podendo ser usado por outras variáveis.

Podemos ter uma coisa chamada **sobrecarga de métodos**, que é **derivado do polimorfismo** (característica que veremos mais à frente, nessa aula). Ele consiste na **definição de diferentes versões de um método na mesma classe, mas com parâmetros diferentes** - então, teremos uma pluralidade de funções com mesmo nome, dentro de uma mesma classe, mas com diferentes argumentos.

A sobrecarga de métodos exige que tenhamos linguagens compatíveis com ela - o que não é muito comum. O Python, que é muito cobrado em concursos, não suporta a sobrecarga de métodos por não permitir variáveis com mesmo nome em um objeto.

Vamos fazer uma sobrecarga no nosso código? Vamos atribuir duas funções diferentes ao atributo **ligar**.

Java

```
class Carro extends Veiculo {  
    public Carro(String marca, String modelo, int ano, String cor) {  
        super(marca, modelo, ano, cor);  
    }  
    //primeira função do método ligar  
    public void ligar() {  
        System.out.println("Carro ligado.");  
    }  
    //segunda função do método ligar  
    public void ligar(boolean ligarLuzes) {  
        if (ligarLuzes) {  
            System.out.println("Luzes do carro ligadas.");  
        } else {  
            System.out.println("Luzes do carro desligadas.");  
        }  
    }  
    public void desligar() {  
        System.out.println("Carro desligado.");  
    }  
}
```

Aqui, o compilador irá executar o ligar duas vezes - na primeira, irá retornar a mensagem "Carro Ligado", e na segunda irá executar uma condicional. Se ligarLuzes estiver com o valor **true**, retornará "Luzes do carro ligadas", caso contrário, retornará "Luzes do carro desligadas".

(FCC/COPERGÁS/2023) A sobrecarga de métodos ocorre quando

- a) um método de mesmo nome existe em classes diferentes em uma relação de herança.
- b) há mais de um método com mesmo nome, recebendo parâmetros de tipos ou em quantidades diferentes, em uma classe.
- c) um método recebe mais parâmetros do que pode suportar.
- d) o mesmo método aparece na maioria das classes da aplicação.
- e) um método recebe como parâmetro um valor que extrapola a capacidade do tipo usado para o recebimento.

Comentários:

Como vimos, sobrecarga de métodos ocorre quando temos mais de uma função com mesmo nome, dentro de uma mesma classe, mas com parâmetros diferentes. (Gabarito: Letra B)

Características da POO

As características da POO que veremos são 5: **abstração, interface, encapsulamento, herança e polimorfismo**. Preste bastante atenção que são elas as responsáveis por trazer funcionalidade para nossa programação, e são **extremamente** cobradas pelas bancas de concurso.

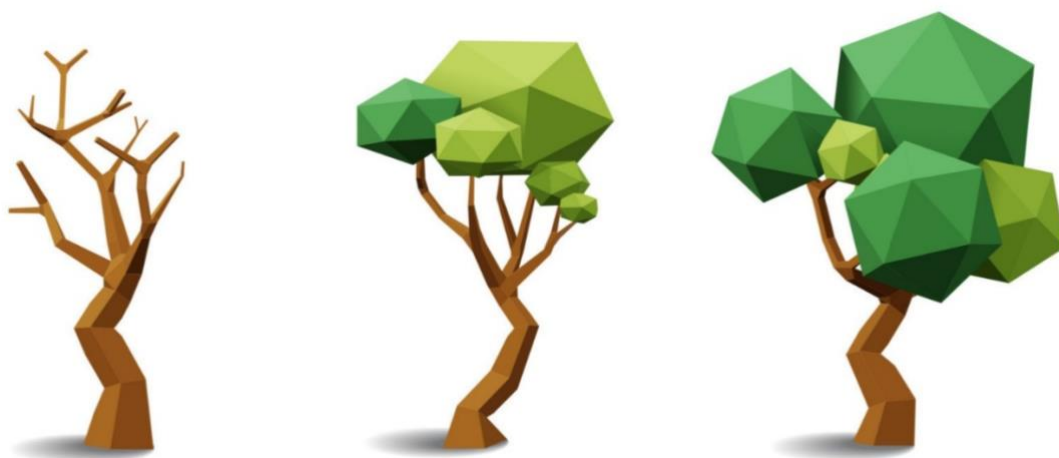
Abstração

Abstração é a **capacidade de se concentrar nas informações relevantes de um objeto, ignorando detalhes irrelevantes ou desnecessários**.

Em outras palavras, a abstração é a simplificação do mundo real em termos de um modelo de objeto que inclui **apenas as informações e funcionalidades relevantes para o programa**. Isso permite que os programadores se concentrem apenas nas características e comportamentos mais importantes do objeto, sem se preocupar com detalhes irrelevantes ou complexos.

A abstração é importante porque ela permite que os programadores criem um modelo de objeto que seja mais fácil de entender e gerenciar do que a complexidade do mundo real. Isso torna mais fácil para os programadores criarem e manter programas orientados a objetos.

Pense em um **jogo de computador**. Quando estamos criando modelos de árvores, usualmente temos muito polígonos (formas geométricas), o que acaba sobrecarregando o gráfico e exigindo demais de computadores. Para isso, árvores mais distantes não requerem muitos detalhes - então, diminuimos as quantidades de polígonos, as deixando mais "abstratas".



(CEBRASPE/CAU BR/2024) Acerca das linguagens e tecnologias de programação, julgue o item subsequente.

Na programação orientada a objetos, a abstração é a característica que permite a utilização de um código ou função sem a necessidade de conhecer detalhes sobre sua implementação.

Comentários:

A abstração foca em ocultar detalhes complexos - somente mostrando-os para quem tem a necessidade de seu conhecimento. Portanto, correto o item. *(Gabarito: Certo)*

Interface

Interface é um **conjunto de métodos e constantes abstratas que define um contrato para um objeto em um programa orientado a objetos**. Uma interface é como uma classe abstrata que não tem implementação, mas define apenas os métodos que devem ser implementados por uma classe que a implementa.

Uma interface define o que um objeto pode fazer, sem se preocupar com a implementação específica de como essa funcionalidade é alcançada. Por exemplo, uma interface "Carro" pode definir métodos como "acelerar", "frear" e "ligar", que qualquer classe que implementa essa interface deve ter, mas a implementação específica desses métodos pode variar de acordo com a classe.

As interfaces permitem que várias classes implementem a mesma interface, o que permite que objetos de diferentes classes sejam usados de forma intercambiável em um programa. Isso permite que os programadores criem código mais genérico, flexível e extensível.

Outro uso comum de interfaces é a implementação de "polimorfismo", que é a capacidade de objetos de diferentes classes serem tratados como se fossem da mesma classe (não se preocupe que veremos esse assunto logo em seguida). Por exemplo, uma classe "Pessoa" pode ter um método "dirigir" que recebe como entrada um objeto que implementa a interface "Carro", permitindo que diferentes tipos de carros sejam usados como parâmetros para esse método.

(FCC/TRT 14/2023) Na programação orientada a objetos, uma interface

- a) pode ser instanciada, desde que tenha um construtor explícito.
- b) deve conter pelo menos um método privado e abstrato.
- c) é uma classe que permite a sobrecarga de construtores.
- d) pode conter assinaturas de métodos, ou seja, métodos que não possuem corpo.
- e) deve possuir todos os seus métodos protegidos (protected).

Comentários:

Vamos analisar cada uma das afirmativas:

- a) Errado. A interface não pode ser instanciada, independentemente de terem um construtor explícito ou não.
- b) Errado. Uma interface não é obrigada a conter métodos privados e abstratos - ela pode ter só privados, só abstratos.
- c) Errado. Interface e classes são coisas diferentes.
- d) Certo. As interfaces contêm assinaturas de métodos.
- e) Errado. A interface não define os níveis de acesso.

Portanto, correta a letra D. (Gabarito: Letra D)

Encapsulamento

O encapsulamento consiste em **ocultar a complexidade interna de um objeto e expor apenas uma interface simples** e bem definida para interagir com ele. Em outras palavras, o encapsulamento é uma técnica que protege os dados e comportamentos de um objeto de acesso externo não autorizado ou indevido, permitindo que o objeto possa ser usado de forma segura e consistente em um programa.

O encapsulamento é alcançado por meio da definição de visibilidade dos membros de uma classe, como atributos e métodos. Ao encapsular um objeto, os programadores podem proteger seus dados e comportamentos de modificações acidentais ou não autorizadas. Isso permite que os objetos sejam usados de forma segura e consistente em um programa, sem interferências externas indesejadas.

Além disso, o **encapsulamento também ajuda a promover a modularidade** e a reutilização de código, permitindo que os objetos sejam desenvolvidos de forma independente e com pouca dependência de outros objetos.

(CEBRASPE/FUB/2022) Julgue o item a seguir, a respeito de projeto e desenvolvimento de sistemas.

O encapsulamento em Java consiste no princípio pelo qual um conjunto de objetos derivados de uma mesma superclasse pode invocar métodos que tenham a mesma identificação (assinatura), mas apresentem comportamentos distintos.

■
Comentários:

Na verdade, o conceito narrado é o de polimorfismo – nosso próximo assunto. *(Gabarito: Errado)*

(CEBRASPE/STM/2018) Julgue o item seguinte, a respeito de programação orientada a objetos.

O encapsulamento permite que um programa seja dividido em várias partes menores; contudo, as partes tornam-se dependentes umas das outras em relação à implementação e em relação ao trabalho realizado.

Comentários:

O encapsulamento “oculta” o interior da classe/objeto a quem não tiver acesso e/ou interesse nesse interior, ele não faz nenhuma divisão. *(Gabarito: Errado)*

Polimorfismo

O polimorfismo permite que objetos **de diferentes classes possam ser tratados de forma intercambiável**, como se fossem da mesma classe. Em outras palavras, o **polimorfismo permite que um objeto possa ser utilizado como se fosse um objeto de outra classe relacionada, desde que atenda a um conjunto de requisitos definidos por uma interface comum**. Isso significa que objetos de diferentes classes podem ser passados como parâmetros para métodos ou funções que esperam um tipo de objeto específico, e ainda assim executar a funcionalidade desejada.

O polimorfismo é alcançado por meio da utilização de classes e interfaces. Quando várias classes implementam a mesma interface, elas podem ser tratadas como se fossem da mesma classe, permitindo que objetos de diferentes classes possam ser usados como se fossem objetos de uma única classe. Isso ajuda a criar um código mais genérico, flexível e extensível.

O **polimorfismo também pode ser alcançado por meio da sobrecarga de métodos**, onde um método pode ter vários comportamentos diferentes, dependendo dos parâmetros de entrada. Isso permite que um único método possa lidar com diferentes tipos de objetos e executar a funcionalidade adequada para cada um deles. Nesse caso, dizemos que temos um *polimorfismo estático*.

(FCC/TRF 3/2019) O Polimorfismo, um dos Pilares da Programação Orientada a Objetos – POO,

- a) ocorre quando uma classe tem um relacionamento do tipo “1 para” com outra classe e isso implica no modo como a definição das classes devem ocorrer nas aplicações.
- b) consiste em esconder os atributos da classe de quem for utilizá-la. Isso se deve a: 1 – para quem for usar a classe não a use de forma errada; e 2 – para que implementação seja feita por meio dos métodos get e set.
- c) permite que um mesmo método possa ter vários comportamentos e a definição de qual comportamento será executado se dá pelo valor diferente de um de seus atributos.
- d) é um conceito que permite que as características bem como as operações, de um modo global, possam ser repassadas para várias funcionalidades da aplicação.
- e) permite utilizar atributos e operações diferentes de uma subclasse, acrescentando ou substituindo características herdadas da classe pai.

Comentários:

O Polimorfismo impõe, basicamente, que um mesmo método possa ter vários comportamentos diferentes, dependendo do objeto (e dos seus respectivos atributos) em que ele está sendo executado. Por exemplo, o método “acelerar” pode ser usado em um objeto carro e um objeto moto, e em cada um ele terá ações diferentes. A afirmativa que melhor representa isso é a letra C. (Gabarito: Letra C)

Herança

Nossa última característica, a **herança permite que uma classe "herde" atributos e métodos de outra classe**. A classe que herda é chamada de classe filha ou subclasse, e a classe que é herdada é chamada de classe pai ou superclasse. A herança pode ser utilizada para definir **relações entre classes**, como generalização e especialização. A generalização é o processo de abstração de características comuns entre várias classes para criar uma classe mais genérica.

Por exemplo, imagine que você está modelando uma aplicação que gerencia veículos. Você pode criar uma classe "Veículo" como uma classe genérica, que inclui atributos e métodos comuns a todos os tipos de veículos, como número de rodas, capacidade do tanque de combustível, etc. Em seguida, você pode criar classes mais específicas, como "Carro", "Moto" e "Caminhão", que herdam os atributos e métodos da classe "Veículo" e adicionam suas próprias características exclusivas.

Já a especialização é o processo de refinamento de uma classe genérica em classes mais específicas. Por exemplo, imagine que você tem uma classe "Animal" que inclui atributos e métodos comuns a todos os animais, como nome, idade, peso, etc. Em seguida, você pode criar classes mais específicas, como "Cachorro", "Gato" e "Pássaro", que herdam os atributos e métodos da classe "Animal" e adicionam suas próprias características exclusivas, como latido, miado ou capacidade de voar.

A herança permite que as classes filhas possam reutilizar o código da classe pai, o que ajuda a criar um código mais organizado e fácil de manter. Além disso, a herança também ajuda a promover a modularidade e a reutilização de código em um programa.

Vamos fazer um *resumão* com o apanhado das características.

Característica de O.O.	Descrição
Abstração	Processo de ocultar detalhes complexos e expor apenas os aspectos essenciais de um objeto, permitindo representar conceitos de forma simplificada.
Interface	Define um contrato que uma classe deve seguir, especificando métodos sem implementar sua lógica, permitindo polimorfismo e desacoplamento.
Encapsulamento	Restrição do acesso direto aos atributos de um objeto, permitindo que apenas métodos controlados possam manipulá-los, garantindo maior segurança e modularidade.
Polimorfismo	Capacidade de um método ou função ter diferentes comportamentos dependendo do contexto, permitindo reutilização de código e flexibilidade na programação.
Herança	Mecanismo que permite que uma classe herde atributos e métodos de outra, promovendo reutilização e hierarquia entre classes.

(FCC/TRF 3/2019) O sistema de controle automotivo SisAut foi modelado orientado a objetos. O SisAut possui a classe Veículo, que compartilha seu código-fonte com suas subclasses: Carro e Moto. As subclasses Carro e Moto aproveitam os métodos e atributos da classe Veículo.

Em orientação a objeto, o mecanismo pelo qual uma classe pode estender outra classe ou ser estendida por outra classe é:

- a) interface;
- b) herança;
- c) pacotes;
- d) agregação;
- e) encapsulamento.

Comentários:

Carro e Moto são especializações de veículos – a característica que permite que “subclasses” aproveitem atributos e métodos da sua classe “pai” é a herança. (Gabarito: Letra B)

QUESTÕES COMENTADAS

01. (CEBRASPE/CNPq/2024) Julgue o próximo item, no que se refere à arquitetura de sistemas.

Na arquitetura orientada a objeto, os dados e as operações são encapsulados para facilitar a manipulação das informações.

Comentários:

Perfeito! O *encapsulamento*, uma das propriedades da Orientação a Objetos, permite que ocultemos as complexidades internas do código, fazendo com que interfaces e classes externas tenham uma interação mais simples com a classe. Assim, podemos concentrar a manipulação das informações internamente.

Gabarito: Certo

02. (CEBRASPE/CAU BR/2024) Acerca das linguagens e tecnologias de programação, julgue o item subsequente.

Na programação orientada a objetos, a abstração é a característica que permite a utilização de um código ou função sem a necessidade de conhecer detalhes sobre sua implementação.

Comentários:

O item está correto. A abstração é um dos pilares da POO e permite que utilizemos classes, métodos ou funções sem a necessidade de conhecer os detalhes internos da implementação. Isso ocorre porque a abstração define apenas os comportamentos essenciais, ocultando a complexidade e garantindo um uso mais intuitivo do código. Um exemplo prático é a interação com uma interface em Java: ao utilizar métodos de uma interface, não precisamos saber como cada classe concreta os implementa, apenas que eles existem e cumprem um contrato.

Gabarito: Certo

03. (CEBRASPE/CAU BR/2024) Quanto ao ciclo de vida do *software* e aos sistemas orientados a objetos, julgue o item subsequente.

Em sistemas orientados a objetos, os objetos podem ser de natureza física, por exemplo, cadeira, ou de natureza conceitual, por exemplo, inscrição em um curso.

Comentários:

Certo! Em POO, um objeto pode representar tanto entidades físicas quanto conceituais. Objetos físicos, como "cadeira" ou "carro", correspondem a elementos do mundo real que podem ser modelados com atributos e comportamentos. Já os objetos conceituais, como "inscrição em um curso" ou "transação bancária", representam entidades abstratas que fazem parte do domínio do sistema. Essa flexibilidade é uma das vantagens da Orientação a Objetos, permitindo modelar sistemas de forma mais próxima da realidade.

04. (CEBRASPE/TST/2024) Na orientação a objetos, os conceitos de superclasse e subclasse estão relacionados a

- a) interface.
- b) polimorfismo.
- c) array.
- d) herança.
- e) coleção.

Comentários:

Vamos analisar as alternativas, procurando aquela que se alinha à especificação de superclasses e subclasses.

- a) Errado. Embora interfaces possam ser usadas em conjunto com herança, elas não definem diretamente a relação entre superclasse e subclasse. Interfaces estabelecem contratos de comportamento que as classes devem seguir, mas não representam uma hierarquia de classes com atributos e métodos herdáveis.
- b) Errado. O polimorfismo permite que um mesmo método tenha diferentes comportamentos em subclasses ou que objetos sejam tratados como instâncias de suas superclasses. No entanto, ele é uma consequência da herança, e não o conceito que define a relação entre superclasse e subclasse.
- c) Errado. Arrays são estruturas de dados usadas para armazenar coleções de elementos do mesmo tipo, mas não têm relação direta com a hierarquia de classes.
- d) Certo. A herança é o conceito da Orientação a Objetos que estabelece a relação entre superclasse (classe base) e subclasse (classe derivada). A subclasse herda atributos e métodos da superclasse, podendo sobrescrevê-los ou adicionar novos comportamentos. Esse mecanismo permite o reuso de código e a especialização de classes.
- e) Errado. Coleções são estruturas utilizadas para armazenar múltiplos objetos de forma flexível, mas não estão diretamente relacionadas ao conceito de superclasse e subclasse. ~

Portanto, correta a letra D.

05. (CEBRASPE/FINEP/2024) Em orientação a objetos, o mecanismo de se criar novas classes a partir de uma classe existente é denominado

- a) herança.
- b) domínio.
- c) polimorfismo.
- d) encapsulamento.
- e) abstração.

Comentários:

Vamos analisar as alternativas.

- a) Certo. Herança é o mecanismo da Orientação a Objetos que permite a criação de novas classes a partir de uma classe existente. A classe original, chamada de superclasse, fornece atributos e métodos que podem ser reutilizados ou sobrescritos pelas subclasses, promovendo a reutilização de código e a especialização de comportamentos.
- b) Errado. O termo "domínio" refere-se ao contexto ou área de aplicação do sistema, mas não tem relação com a criação de novas classes a partir de uma existente.
- c) Errado. O polimorfismo permite que métodos tenham diferentes implementações dependendo da classe que os define, mas não está diretamente relacionado à criação de classes a partir de outras.
- d) Errado. O encapsulamento restringe o acesso direto aos atributos e métodos de uma classe, protegendo seus dados, mas não trata da criação de novas classes a partir de uma existente.
- e) Errado. A abstração define apenas os detalhes essenciais de uma classe e oculta sua complexidade, mas não está relacionada à herança de atributos e métodos.

Portanto, correta a letra A.

Gabarito: Letra A

06. (CEBRASPE/LNA/2024)

```
class carro:
    def __init__(self, nome):
        self.nome = nome

    def acao(self):
        print("corre")

class aviao:
    def __init__(self, nome):
        self.nome = nome

    def acao(self):
        print("voa")

objetoA = carro("x3")
objetoB = aviao("f15")

print("\nAção:")
for x in (objetoA, objetoB):
    x.acao()
```

A seguir, é apresentado o resultado do código precedente, desenvolvido em Python, ao final de sua execução.

Ação:
corre
voa

Com base nessas informações, é correto afirmar que o princípio da orientação a objeto utilizado nesse programa, especialmente em relação ao método acao, é a(o)

- a) abstração.
- b) encapsulamento.
- c) herança.
- d) polimorfismo.
- e) recursividade.

Comentários:

Apesar do código estar em Python, acredito que você consiga entender a questão. O código exemplifica polimorfismo, pois os objetos das classes carro e aviao possuem um método com o mesmo nome (acao), mas implementado de maneira distinta em cada classe. O laço for permite que ambos os objetos sejam tratados de forma uniforme, executando o método correspondente à classe específica de cada um. Isso caracteriza polimorfismo, permitindo que diferentes classes compartilhem uma interface comum.

Gabarito: Letra D

07. (CEBRASPE/CAGEPA/2024) Assinale a opção correta a respeito de conceitos pertinentes à orientação a objetos.

- a) Herança representa a capacidade de uma única referência invocar métodos diferentes, dependendo do seu encapsulamento.
- b) Encapsulamento identifica o princípio de ocultar os detalhes internos de uma classe e expor apenas o que é necessário por meio de interfaces públicas.
- c) Polimorfismo é o mecanismo pelo qual uma classe (subclasse) herda características e comportamentos de outra classe (superclasse).
- d) Composição relaciona classes em uma hierarquia, em que uma subclasse herda atributos e métodos de uma superclasse e é utilizada quando há um relacionamento "é-um" (is-a).
- e) Abstração é o mecanismo de ocultar o código fonte e comportamentos de outra classe (superclasse).

Comentários:

Vamos analisar as alternativas.

- a) Errado. O conceito descrito se assemelha mais ao polimorfismo, que permite que uma referência invoque diferentes implementações de um método, dependendo do objeto ao qual está associada. Herança, por outro lado, refere-se ao mecanismo pelo qual uma subclasse herda atributos e métodos de uma superclasse.

- b) Certo. O encapsulamento é um dos princípios fundamentais da Orientação a Objetos e tem como objetivo restringir o acesso direto aos atributos de uma classe, expondo apenas os métodos necessários para interação. Isso promove maior segurança e modularidade no código, permitindo que a implementação interna possa ser alterada sem impactar outras partes do sistema.
- c) Errado. Essa definição corresponde à herança, e não ao polimorfismo. O polimorfismo permite que um mesmo método seja implementado de diferentes formas em classes distintas, promovendo flexibilidade na programação.
- d) Errado. A descrição apresentada corresponde à herança, e não à composição. A composição, de fato, representa um relacionamento entre classes, mas do tipo "tem-um" (has-a), onde uma classe contém instâncias de outra como parte de sua estrutura.
- e) Errado. A abstração trata da modelagem de classes, destacando apenas os aspectos essenciais e ocultando detalhes desnecessários, mas não se limita ao código-fonte ou a uma relação com superclasses.

Correta, portanto, a letra B.

Gabarito: Letra B

08. (CEBRASPE/STJ/2024) Determinado sistema tem duas classes, Carro e Bicicleta. Ambas têm atributos como marca e modelo, além de métodos próprios, mas compartilham uma classe-pai comum de nome Veiculo.

A partir dessa situação hipotética, julgue o item a seguir, a respeito de herança em orientação a objetos.

As classes Carro e Bicicleta fazem parte de uma estrutura de herança hierárquica, na qual os métodos e funcionalidades das subclasses são automaticamente sobrepostos e eliminados pelos da classe-pai Veiculo.

Comentários:

O item está incorreto. As classes Carro e Bicicleta herdam atributos e métodos de uma classe-pai, denominada Veiculo, o que caracteriza uma estrutura de herança hierárquica. No entanto, o enunciado afirma que os métodos e funcionalidades das subclasses são automaticamente sobrepostos e eliminados pela classe-pai, o que está incorreto. Na verdade, ocorre o oposto: a subclasse pode sobrescrever métodos herdados da classe-pai por meio de uma técnica chamada "sobrescrita de métodos" (ou overriding), mas essa sobrescrita não elimina o método da classe-pai.

Assim, as funcionalidades da classe-pai são mantidas e acessíveis, exceto se forem especificamente sobrescritas nas subclasses. Ainda assim, é possível acessar os métodos originais da classe-pai utilizando a referência dela, como super em algumas linguagens de programação.

Gabarito: Errado

09. (CEBRSAPE/EMPREL/2023) Assinale a opção que apresenta os quatro principais conceitos em que a programação orientada a objetos é baseada.

- a) blocos de dados; datafiles; estruturas físicas; tablespaces
- b) abstração; encapsulamento; herança; polimorfismo
- c) chave primária; consulta; projeção; relacionamento
- d) modelo conceitual; modelo lógico; projeção; seleção
- e) índices; relacionamentos; segurança; transações

Comentários:

Questão tranquila – os princípios da Orientação a Objetos são: abstração, encapsulamento, herança e polimorfismo. Correta a letra B.

Gabarito: Letra B

10. (CEBRASPE/EMPREL/2023) Em relação a conceitos de programação orientada a objetos, assinale a opção correta.

- a) Padrões de frontend são protocolos que garantem a segurança do código.
- b) Um método é uma maneira de programar com protocolo próprio independente do programa principal.
- c) Polimorfismo é a capacidade de escrever métodos que se comportem corretamente para objetos de tipos diferentes.
- d) O uso de passagem de parâmetros em um método tem por finalidade efetuar um ponto de comunicação unidirecional entre programas.
- e) O padrão SPA trata múltiplas páginas de aplicação web e garante a segurança da aplicação.

Comentários:

A questão traz alguns conceitos de desenvolvimento web no meio, mas vamos analisar as alternativas.

- a) Errado. Padrões de frontend dizem respeito a boas práticas para o desenvolvimento de interfaces de usuário, como separação de responsabilidades e uso de frameworks apropriados. No entanto, eles não são protocolos nem garantem segurança diretamente, pois a segurança envolve outras medidas, como controle de acesso, autenticação e criptografia.
- b) Errado. Um método é uma função associada a uma classe em programação orientada a objetos, sendo responsável por definir comportamentos para os objetos dessa classe. Ele não possui um "protocolo próprio independente", pois faz parte da estrutura da classe e interage com o restante do código.
- c) Certo. O polimorfismo permite que um mesmo método tenha diferentes implementações dependendo do objeto ao qual está associado. Isso pode ocorrer por meio da sobrescrita de métodos em subclasses ou da implementação de interfaces, tornando o código mais flexível e reutilizável.

- d) Errado. A passagem de parâmetros em um método permite a comunicação entre diferentes partes de um programa, mas não se restringe à comunicação unidirecional entre programas distintos. Os parâmetros podem ser usados para modificar estados internos de objetos, processar dados e retornar valores, sendo um conceito interno à estrutura do código.
- e) Errado. SPA (Single Page Application) refere-se a uma arquitetura de desenvolvimento web na qual a navegação ocorre dentro de uma única página, carregando dinamicamente o conteúdo conforme necessário. Ele não trata de múltiplas páginas e, por si só, não garante segurança, sendo necessário implementar medidas adicionais, como proteção contra ataques XSS e CSRF.

Correta a letra C.

Gabarito: Letra C

11. (CEBRASPE/ISS Fortaleza/2023) A respeito de programação estruturada e orientada a objetos, julgue o item subsequente.

Em programação orientada a objetos, abstração é a propriedade que permite que um método de determinado nome tenha comportamentos distintos, em função de diferentes parâmetros recebidos.

Comentários:

A definição apresentada descreve sobrecarga de métodos, que faz parte do conceito de polimorfismo, e não de abstração. A abstração se encarrega de ocultar detalhes internos de implementação, expondo apenas os aspectos essenciais de um objeto. Isso permite que programadores trabalhem com interfaces simplificadas, sem a necessidade de conhecer a complexidade interna da classe.

Já a situação descrita no item, em que um método pode ter diferentes comportamentos dependendo dos parâmetros recebidos, está relacionada à sobrecarga de métodos, uma forma de polimorfismo estático. Esse mecanismo permite que uma classe defina múltiplos métodos com o mesmo nome, mas assinaturas diferentes.

Gabarito: Errado

12. (CEBRASPE/MPE RO/2023) Em orientação a objetos, o conceito utilizado para descrever os vários comportamentos que um método possui, visando a um melhor aproveitamento de partes de um código, é denominado

- a) herança.
- b) encapsulamento.
- c) atributos.
- d) polimorfismo.
- e) interface.

Comentários:

A questão explora o conceito de *polimorfismo*. Ele permite que um mesmo método tenha diferentes comportamentos dependendo do objeto ao qual está associado ou dos parâmetros recebidos. Isso ocorre, por exemplo, quando sobrescrevemos métodos em subclasses (polimorfismo dinâmico) ou utilizamos métodos com o mesmo nome, mas assinaturas diferentes (polimorfismo estático). Esse princípio melhora a reutilização e flexibilidade do código. Correta a letra D.

Gabarito: Letra D

13. (CEBRASPE/DATAPREV/2023) A respeito de engenharia de software, julgue o item a seguir.

Herança é uma característica do paradigma orientado a objetos, a qual possibilita que haja hierarquia de classes, de forma que as alterações em uma classe-pai possam ser imediatamente propagadas para a classe-filha.

Comentários:

Um pouco “polêmico o item”. A herança é, de fato, um mecanismo da Orientação a Objetos que permite a criação de uma hierarquia de classes, onde uma classe-filha herda atributos e métodos de uma classe-pai. No entanto, a afirmação de que as alterações na classe-pai são imediatamente propagadas para a classe-filha não é correta em todos os casos.

Na prática, quando a classe-pai sofre modificações, a classe-filha não recebe automaticamente essas alterações, especialmente se houver métodos sobrescritos. Se novos métodos ou atributos forem adicionados à classe-pai, a classe-filha só poderá acessá-los caso não haja restrições de visibilidade (como `private`). Além disso, se um método herdado for modificado na classe-pai, pode ser necessário atualizar a implementação na classe-filha para garantir compatibilidade.

Por não dizer que as mudanças serão propagadas automaticamente *sempre*, podemos passar um pano e concordar com o gabarito da banca – que foi *certo*.

Gabarito: Certo

14. (CEBRASPE/DATAPREV/2023) A respeito de engenharia de software, julgue o item a seguir.

No paradigma da orientação a objetos, o polimorfismo permite que várias operações distintas possuam o mesmo nome, desacoplando, assim, os objetos uns dos outros, tornando-os mais independentes.

Comentários:

O item está correto! O polimorfismo é uma característica fundamental da Orientação a Objetos que permite que diferentes classes implementem métodos com o mesmo nome, mas com comportamentos distintos. Isso pode ocorrer de duas formas principais:

- Polimorfismo de sobrecarga (estático): Quando há múltiplos métodos com o mesmo nome dentro da mesma classe, mas com diferentes assinaturas (quantidade ou tipos de parâmetros).
- Polimorfismo de sobrescrita (dinâmico): Quando uma subclasse redefine um método da superclasse, alterando seu comportamento sem modificar sua assinatura.

Esse mecanismo promove o desacoplamento entre objetos, pois permite que um mesmo método seja invocado em diferentes contextos sem a necessidade de saber exatamente qual implementação está sendo utilizada. Isso melhora a extensibilidade e a manutenção do código, tornando os objetos mais independentes entre si.

Gabarito: Certo

15. (CEBRASPE/TC DF/2023) Acerca de orientação a objetos, julgue o item subsequente.

Considere-se que, em um encapsulamento de uma classe de nome Carro para um sistema de automóveis, exista um atributo privado de nome quilometragem. Nesse caso, ao se fornecerem métodos públicos do tipo `obter_quilometragem()` e `atualizar_quilometragem()`, protege-se detalhes internos da classe Carro.

Comentários:

Certo! O encapsulamento é um dos princípios fundamentais da Orientação a Objetos e tem como objetivo proteger os detalhes internos de uma classe, permitindo o acesso e a modificação de atributos apenas por meio de métodos específicos.

No exemplo apresentado, o atributo quilometragem é privado, ou seja, não pode ser acessado diretamente de fora da classe. Para permitir a leitura e a modificação desse atributo sem expor sua implementação interna, são criados métodos públicos como `obter_quilometragem()` (getter) e `atualizar_quilometragem()` (setter).

Dessa forma, evita-se a manipulação direta da quilometragem, garantindo maior controle sobre os dados e permitindo aplicar validações antes de qualquer alteração. Esse mecanismo fortalece a segurança e a integridade dos objetos, além de facilitar futuras mudanças na implementação sem impactar o código que utiliza a classe.

Gabarito: Certo

QUESTÕES COMENTADAS

01. (CEBRASPE/CNPq/2024) Julgue o próximo item, no que se refere à arquitetura de sistemas.

Na arquitetura orientada a objeto, os dados e as operações são encapsulados para facilitar a manipulação das informações.

02. (CEBRASPE/CAU BR/2024) Acerca das linguagens e tecnologias de programação, julgue o item subsequente.

Na programação orientada a objetos, a abstração é a característica que permite a utilização de um código ou função sem a necessidade de conhecer detalhes sobre sua implementação.

03. (CEBRASPE/CAU BR/2024) Quanto ao ciclo de vida do *software* e aos sistemas orientados a objetos, julgue o item subsequente.

Em sistemas orientados a objetos, os objetos podem ser de natureza física, por exemplo, cadeira, ou de natureza conceitual, por exemplo, inscrição em um curso.

04. (CEBRASPE/TST/2024) Na orientação a objetos, os conceitos de superclasse e subclasse estão relacionados a

- a) interface.
- b) polimorfismo.
- c) array.
- d) herança.
- e) coleção.

05. (CEBRASPE/FINEP/2024) Em orientação a objetos, o mecanismo de se criar novas classes a partir de uma classe existente é denominado

- a) herança.
- b) domínio.
- c) polimorfismo.
- d) encapsulamento.
- e) abstração.

06. (CEBRASPE/LNA/2024)

```
class carro:
    def __init__(self, nome):
        self.nome = nome

    def acao(self):
        print("corre")

class aviao:
```

```
def __init__(self, nome):  
    self.nome = nome  
  
def acao(self):  
    print("voa")  
  
objetoA = carro("x3")  
objetoB = aviao("f15")  
  
print("\nAção:")  
for x in (objetoA, objetoB):  
    x.acao()
```

A seguir, é apresentado o resultado do código precedente, desenvolvido em Python, ao final de sua execução.

Ação:
corre
voa

Com base nessas informações, é correto afirmar que o princípio da orientação a objeto utilizado nesse programa, especialmente em relação ao método acao, é a(o)

- a) abstração.
- b) encapsulamento.
- c) herança.
- d) polimorfismo.
- e) recursividade.

07. (CEBRASPE/CAGEPA/2024) Assinale a opção correta a respeito de conceitos pertinentes à orientação a objetos.

- a) Herança representa a capacidade de uma única referência invocar métodos diferentes, dependendo do seu encapsulamento.
- b) Encapsulamento identifica o princípio de ocultar os detalhes internos de uma classe e expor apenas o que é necessário por meio de interfaces públicas.
- c) Polimorfismo é o mecanismo pelo qual uma classe (subclasse) herda características e comportamentos de outra classe (superclasse).
- d) Composição relaciona classes em uma hierarquia, em que uma subclasse herda atributos e métodos de uma superclasse e é utilizada quando há um relacionamento "é-um" (is-a).
- e) Abstração é o mecanismo de ocultar o código fonte e comportamentos de outra classe (superclasse).

08. (CEBRASPE/STJ/2024) Determinado sistema tem duas classes, Carro e Bicicleta. Ambas têm atributos como marca e modelo, além de métodos próprios, mas compartilham uma classe-pai comum de nome Veiculo.

A partir dessa situação hipotética, julgue o item a seguir, a respeito de herança em orientação a objetos.

As classes Carro e Bicicleta fazem parte de uma estrutura de herança hierárquica, na qual os métodos e funcionalidades das subclasses são automaticamente sobrepostos e eliminados pelos da classe-pai Veiculo.

09. (CEBRASPE/EMPREL/2023) Assinale a opção que apresenta os quatro principais conceitos em que a programação orientada a objetos é baseada.

- a) blocos de dados; datafiles; estruturas físicas; tablespaces
- b) abstração; encapsulamento; herança; polimorfismo
- c) chave primária; consulta; projeção; relacionamento
- d) modelo conceitual; modelo lógico; projeção; seleção
- e) índices; relacionamentos; segurança; transações

10. (CEBRASPE/EMPREL/2023) Em relação a conceitos de programação orientada a objetos, assinale a opção correta.

- a) Padrões de frontend são protocolos que garantem a segurança do código.
- b) Um método é uma maneira de programar com protocolo próprio independente do programa principal.
- c) Polimorfismo é a capacidade de escrever métodos que se comportem corretamente para objetos de tipos diferentes.
- d) O uso de passagem de parâmetros em um método tem por finalidade efetuar um ponto de comunicação unidirecional entre programas.
- e) O padrão SPA trata múltiplas páginas de aplicação web e garante a segurança da aplicação.

11. (CEBRASPE/ISS Fortaleza/2023) A respeito de programação estruturada e orientada a objetos, julgue o item subsequente.

Em programação orientada a objetos, abstração é a propriedade que permite que um método de determinado nome tenha comportamentos distintos, em função de diferentes parâmetros recebidos.

12. (CEBRASPE/MPE RO/2023) Em orientação a objetos, o conceito utilizado para descrever os vários comportamentos que um método possui, visando a um melhor aproveitamento de partes de um código, é denominado

- a) herança.
- b) encapsulamento.
- c) atributos.
- d) polimorfismo.
- e) interface.

13. (CEBRASPE/DATAPREV/2023) A respeito de engenharia de software, julgue o item a seguir.

Herança é uma característica do paradigma orientado a objetos, a qual possibilita que haja hierarquia de classes, de forma que as alterações em uma classe-pai possam ser imediatamente propagadas para a classe-filha.

14. (CEBRASPE/DATAPREV/2023) A respeito de engenharia de software, julgue o item a seguir.

No paradigma da orientação a objetos, o polimorfismo permite que várias operações distintas possuam o mesmo nome, desacoplando, assim, os objetos uns dos outros, tornando-os mais independentes.

15. (CEBRASPE/TC DF/2023) Acerca de orientação a objetos, julgue o item subsequente.

Considere-se que, em um encapsulamento de uma classe de nome Carro para um sistema de automóveis, exista um atributo privado de nome quilometragem. Nesse caso, ao se fornecerem métodos públicos do tipo `obter_quilometragem()` e `atualizar_quilometragem()`, protegem-se detalhes internos da classe Carro.

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.